SafeToon: Deep Learning-Based Child Content Safety System

Project Team

Muhammad Mubeen 21i-0794
Haider Rizwan 22i-2379

Session 2021-2026

Supervised by

Dr. Muhammad Rizwan

Co-Supervised by

Dr. Afia Zafar

Department of Computer Science

National University of Computer and Emerging Sciences
Islamabad, Pakistan

June, 2026

Contents

List of Figures

List of Tables

Chapter 1

Introduction

This document presents the midterm report for SafeToon, a deep learning-based child content safety system designed to detect and mitigate inappropriate content in animated videos. The system addresses the growing concern of "Elsagate" content—disturbing videos disguised as child-friendly cartoons featuring popular characters like Elsa, Spiderman, and Mickey Mouse, but containing violence, sexual themes, profanity, and psychologically harmful material [3]. SafeToon implements a comprehensive pipeline that processes video and audio streams in parallel, classifies content using deep learning models inspired by AnimeNet [4] and Ishikawa et al. [1], and applies a "Surgical Cut" method to blur, mute, or skip inappropriate segments while preserving the narrative flow of safe content.

1.1 Problem Statement

Children today consume vast amounts of video content through streaming platforms (YouTube, Netflix), television broadcasts, and downloaded media. A significant portion of animated content, while appearing child-friendly on the surface, contains inappropriate material including graphic violence, sexual innuendo, profanity, and disturbing behavioral patterns. This phenomenon, termed "Elsagate," exploits familiar cartoon aesthetics to bypass parental vigilance and platform moderation algorithms. Current solutions are inadequate: platform filters rely on metadata that can be manipulated, parental controls offer only binary block/allow options, and existing research prototypes focus solely on classification without providing actionable mitigation. Parents lack tools to pre-screen content or automatically sanitize videos before their children watch them. The consequence is direct exposure of young viewers to content that negatively impacts psychological development, behavior, and emotional well-being. There is an urgent need for a software system that can analyze both visual and audio content of animated videos, identify inappropriate seg-

ments with severity grading, and automatically censor harmful portions while preserving the viewable, age-appropriate content.

1.2 Motivation

The motivation for SafeToon stems from multiple converging factors:

1. **Prevalence of Elsagate Content:** Research indicates that disturbing content disguised as children's entertainment continues to proliferate across video platforms, with creators deliberately mimicking child-friendly aesthetics to evade detection [2].

2. **Inadequacy of Existing Solutions:** Current platform filters (YouTube Kids, parental controls) have repeatedly failed to prevent inappropriate content from reaching children, relying on easily-manipulated metadata rather than deep content analysis.

3. **Research-to-Practice Gap:** While academic work has proposed detection models (AnimeNet, Ishikawa et al.) no complete, deployable system exists that parents can actually use.

4. **Multi-Source Content Consumption:** Children watch content from multiple sources—streaming platforms, downloaded videos, and television broadcasts. A comprehensive solution must handle content regardless of source.

1.3 Problem Solution

SafeToon provides a complete software solution for detecting and mitigating inappropriate content in animated videos through the following approach:

Objectives:

1. Develop a frame classification module using CNN-based deep learning (inspired by AnimeNet) to detect violence and erotic content at multiple severity levels (Low, Medium, High).

2. Implement an audio analysis pipeline using speech-to-text transcription and NLP-based profanity/emotion detection to identify inappropriate audio content.

3. Create the "Surgical Cut" engine that applies targeted censorship (frame blurring, audio muting, segment skipping) to flagged content while preserving safe material.

4. Build a user-friendly desktop application with a parent dashboard for uploading videos, configuring sensitivity policies, and viewing analysis reports.
5. Support multiple input sources including video files (MP4, AVI, MKV), and image sequences.
6. Implement efficient parallel processing architecture to minimize processing time—targeting processing speeds and faster playback duration for practical usability.
7. Generate comprehensive content reports with timestamped logs of all detected inappropriate segments and applied censorship actions.

Goals:

- Achieve frame classification accuracy of $\geq$75%
- Achieve profanity detection accuracy of $\geq$90%
- Process video content faster than real-time (target: 2-3x playback speed)
- Deliver a functional, user-tested application by project completion

1.4 Scope

SafeToon is a **desktop application** designed to analyze and sanitize animated video content for child safety. The system accepts video files as input, processes both visual frames and audio tracks in parallel using deep learning models, classifies content into severity categories, and applies the "Surgical Cut" method to produce a child-safe output video. The application targets parents who want to pre-screen content before allowing their children to watch it. The core functionalities include:

1. Video ingestion from files (MP4, AVI, MKV).
2. Frame extraction and CNN-based classification for violence and erotic content at three severity levels.
3. Audio extraction, speech-to-text transcription, and NLP-based profanity/emotion detection.
4. Surgical Cut processing that blurs flagged frames, mutes profane audio segments, or skips severely inappropriate sections.
5. Output generation of sanitized video files.

6. Parent dashboard for configuration and report viewing.

Scope Boundaries (What SafeToon DOES):

- Processes pre-recorded video files (batch processing mode)
- Classifies frames into 7 categories (Violence L1/L2/L3, Erotic L1/L2/L3, Normal)
- Detects profanity and aggressive emotions in audio
- Applies blur, mute, or skip censorship based on user-defined sensitivity
- Generates sanitized output video and detailed content reports
- Provides desktop GUI for parent interaction

Scope Boundaries (As per the scope of FYP, What SafeToon does NOT do):

- Does NOT provide live/real-time streaming analysis (processes recorded content)
- Does NOT integrate directly with YouTube/Netflix APIs
- Does NOT include mobile application (desktop only)
- Does NOT perform scene reconstruction or AI-generated content replacement
- Does NOT require cloud connectivity (runs locally on user's machine)

Addressing Panel Concern - Processing Time: SafeToon implements efficient parallel processing where frame analysis and audio analysis run concurrently. With GPU acceleration and optimized inference, the system targets processing speeds of 2-3x faster than real-time playback. A 30-minute cartoon episode would require approximately 10-15 minutes to process, NOT equal-duration processing.

Addressing Panel Suggestion - Television Source: After discussion with our supervisor and considering that Team SafeToon consists of only two members with an already substantial project scope, we have decided not to include television as an input source in the current phase. However, once the core solution is built and validated, extending support to television sources will be a natural future enhancement.

1.5 Modules

The system is decomposed into six functional modules, each responsible for a distinct stage of the processing pipeline.

1.5.1 Module 1: Video Ingestion & Preprocessing

This module handles input acquisition from multiple input formats and prepares content for analysis. It separates video into individual frames and extracts the audio track for parallel processing. **This module was fully implemented in Iteration 1.**

1. Accept video file uploads (MP4, AVI, MKV, WebM formats)
2. Extract frames at configurable rate (default: 1 frame per second for efficiency)
3. Extract audio track and segment into processing chunks
4. Queue frames and audio for parallel processing pipelines

1.5.2 Module 2: Frame Analyzer (Visual Classification)

This module implements the deep learning model for visual content classification, inspired by AnimeNet's architecture with channel-spatial attention mechanisms [4].

1. Load and preprocess frames (resize to 224x224 for model input)
2. Run CNN inference for 7-class classification (Violence L1/L2/L3, Erotic L1/L2/L3, Normal)
3. Apply skip-frame strategy (analyze every 3rd frame) for efficiency
4. Generate frame-level classification results with confidence scores
5. Support GPU acceleration via PyTorch CUDA for faster inference

1.5.3 Module 3: Audio Analyzer

This module processes the audio track to detect profanity, inappropriate language, and emotional tone that may indicate harmful content.

1. Transcribe speech to text using Whisper AI
2. Detect profanity using keyword matching against curated lexicon
3. Classify emotional tone (angry, fearful, neutral, happy) using sentiment analysis
4. Generate timestamped audio classification results
5. Apply Voice Activity Detection (VAD) to skip silence/music-only segments

1.5.4 Module 4: Surgical Cut Engine

This is the core mitigation module that applies censorship to flagged content based on user-defined sensitivity policies.

1. Receive classification results from Frame Analyzer and Audio Analyzer
2. Apply user-defined threshold rules (e.g., blur Violence L2+, mute all profanity)
3. Execute frame blurring using Gaussian blur filter on flagged visual content
4. Execute audio muting by zeroing samples in profane segments
5. Optionally skip severely inappropriate segments entirely
6. Reconstruct video with censored frames and audio

1.5.5 Module 5: Output Generator

This module produces the final sanitized video and comprehensive analysis reports.

1. Combine censored video frames and audio into output video file
2. Maintain audio-video synchronization
3. Generate detailed content report (PDF/JSON) with timestamps of all flagged content
4. Provide summary statistics (total flags by category, percentage safe content)
5. Export to common video formats (MP4 with H.264 encoding)

1.5.6 Module 6: Parent Dashboard (Desktop GUI)

This module provides the user interface for parents to interact with the system.

1. Video upload interface with drag-and-drop support
2. Processing queue management with progress indicators
3. Sensitivity configuration panel (per-category threshold sliders)
4. Report viewer with interactive timeline showing flagged segments
5. Video player for previewing original and censored output
6. Settings management (output directory, default policies)

1.6 Stakeholders

The key stakeholders for SafeToon are:

1. **Parents/Guardians:** Primary users who need to pre-screen and sanitize video content before their children watch it. They require an easy-to-use interface with configurable sensitivity settings.
2. **Children (Ages 3-13):** Indirect beneficiaries who will be protected from exposure to inappropriate content. The system must preserve entertainment value of safe content.
3. **Educational Institutions:** Schools and daycare centers that show video content to children need assurance that material is age-appropriate.
4. **Content Creators:** Animation studios and content platforms may use such technology to pre-screen their content before publication.
5. **Child Safety Organizations:** NGOs and regulatory bodies focused on child protection can use the system for content auditing and research.
6. **Academic Community:** Researchers in computer vision, NLP, and child safety can build upon this implemented system for further studies.

Chapter 2

Project Requirements

This chapter describes the functional and non-functional requirements of SafeToon, derived from the system scope and stakeholder analysis presented in Chapter 1.

2.1 Use Cases

The primary user interactions with SafeToon are captured through use case modeling. Figure 2.1 presents the use case diagram.

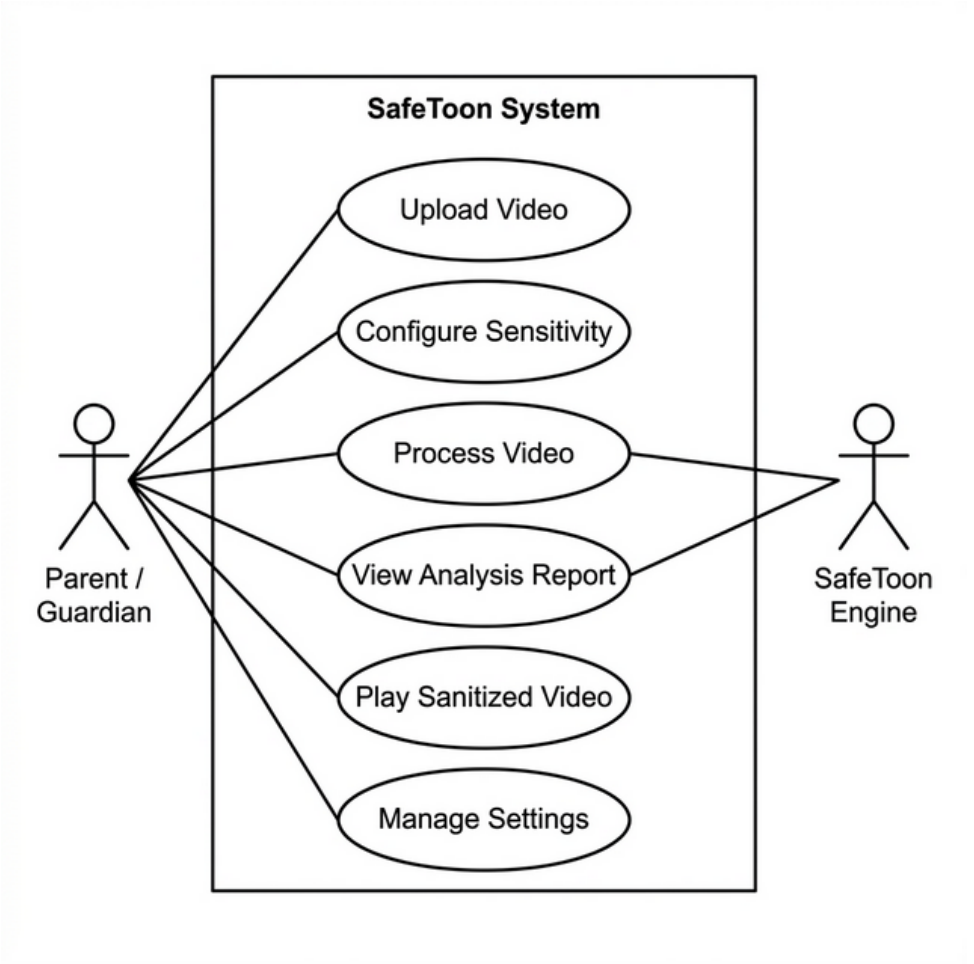

Figure 2.1: Use Case Diagram

2.1.1 Detailed Use Case: Process Video

Table 2.1: Detailed Use Case – Process Video

Field	**Description**
Use Case ID	UC-03
Use Case Name	Process Video
Primary Actor	Parent / Guardian
Secondary Actor	SafeToon Engine
Preconditions	A valid video file has been uploaded (UC-01). Sensitivity thresholds have been configured (UC-02, or defaults apply).
Main Flow	1. Parent clicks "Process" on an uploaded video. 2. System validates the video file (format, size, readability). 3. System assigns a unique Job ID and creates output directories. 4. System extracts video metadata (duration, FPS, resolution, audio presence). 5. System extracts frames at the configured rate (default: 1 FPS). 6. System extracts and segments the audio track into chunks. 7. Frame Analyzer classifies each frame (Violence / Erotic / Normal). 8. Audio Analyzer transcribes speech and detects profanity and emotional tone. 9. Surgical Cut Engine applies censorship (blur / mute / skip) per sensitivity policy. 10. Output Generator reconstructs the sanitized video and generates the analysis report. 11. System notifies the parent that processing is complete.
Postconditions	A sanitized video file and a detailed content report (with timestamps of all flagged segments) are available for viewing.
Alternative Flows	• **2a.** Validation fails → system displays error with reason; processing aborts. • **6a.** No audio track present → audio analysis is skipped; only visual classification is applied. • **9a.** All frames classified as Normal → original video is passed through unchanged; report shows 100% safe.

2.1.2 Detailed Use Case: Upload Video

Table 2.2: Detailed Use Case – Upload Video

Field	**Description**
Use Case ID	UC-01
Use Case Name	Upload Video
Primary Actor	Parent / Guardian
Preconditions	SafeToon application is running.
Main Flow	1. Parent opens the Dashboard and clicks "Upload Video" or drags a file into the upload zone. 2. System validates the file (checks extension, file size limit of 500 MB, and FFmpeg readability). 3. System adds the video to the processing queue with a "Ready" status. 4. Dashboard displays the video entry with filename, size, and status indicator.
Postconditions	Video is queued and ready for processing.
Alternative Flows	• **2a.** Unsupported format → error message displayed; file not added. • **2b.** File exceeds size limit → error message with current limit shown.

2.2 Functional Requirements

2.2.1 Module 1: Video Ingestion & Preprocessing

Following are the functional requirements for the Video Ingestion module:

1. **FR-1.1:** The system shall accept video files in MP4, AVI, MKV, and WebM formats.
2. **FR-1.2:** The system shall validate each uploaded file for existence, supported extension, file size ($\leq$500 MB), and FFmpeg readability before processing.
3. **FR-1.3:** The system shall assign a unique UUID-based job identifier to each ingested video and create structured output directories (`frames/`, `audio/`, `metadata.json`).
4. **FR-1.4:** The system shall extract video metadata including duration, FPS, resolution, and audio presence using FFprobe.
5. **FR-1.5:** The system shall extract video frames at a user-configurable rate (default: 1 frame per second) using OpenCV.
6. **FR-1.6:** The system shall extract the audio track to 16 kHz mono WAV format using FFmpeg.

7. **FR-1.7:** The system shall segment extracted audio into fixed-length chunks (default: 10 seconds) for downstream analysis.
8. **FR-1.8:** The system shall persist job state and metadata as structured JSON for traceability.

2.2.2 Module 2: Frame Analyzer

Following are the functional requirements for the Frame Analyzer module:

1. **FR-2.1:** The system shall preprocess extracted frames by converting BGR→RGB, resizing to 224×224 pixels, and applying ImageNet normalization.
2. **FR-2.2:** The system shall classify each preprocessed frame into one of three categories: Violence, Erotic, or Normal.
3. **FR-2.3:** The system shall output a confidence score (probability) for each classification.
4. **FR-2.4:** The system shall utilize GPU acceleration (CUDA) when available to improve inference throughput.

2.2.3 Module 3: Audio Analyzer

Following are the functional requirements for the Audio Analyzer module:

1. **FR-3.1:** The system shall transcribe speech from audio chunks using the Whisper AI model.
2. **FR-3.2:** The system shall detect profanity by matching transcribed text against a curated lexicon.
3. **FR-3.3:** The system shall classify the emotional tone of each audio segment (angry, fearful, neutral, happy).
4. **FR-3.4:** The system shall generate timestamped results linking profanity and emotion flags to specific audio segments.

2.2.4 Module 4: Surgical Cut Engine

Following are the functional requirements for the Surgical Cut Engine:

1. **FR-4.1:** The system shall apply Gaussian blur to video frames classified as inappropriate above the user-defined sensitivity threshold.
2. **FR-4.2:** The system shall mute audio segments containing detected profanity by zeroing the audio samples.
3. **FR-4.3:** The system shall optionally skip (remove) video segments classified as severely inappropriate.

4. **FR-4.4:** The system shall maintain frame-accurate audio-video synchronization after censorship is applied.

2.2.5 Module 5: Output Generator

Following are the functional requirements for the Output Generator:

1. **FR-5.1:** The system shall reconstruct the sanitized video in MP4 format with H.264 encoding.
2. **FR-5.2:** The system shall generate a content analysis report containing timestamps, categories, and confidence scores for all flagged segments.
3. **FR-5.3:** The system shall provide summary statistics including total flags per category and overall safe-content percentage.

2.2.6 Module 6: Parent Dashboard

Following are the functional requirements for the Parent Dashboard:

1. **FR-6.1:** The system shall provide a graphical upload interface with drag-and-drop support.
2. **FR-6.2:** The system shall display a processing queue with real-time progress indicators.
3. **FR-6.3:** The system shall allow parents to configure sensitivity thresholds per content category.
4. **FR-6.4:** The system shall display analysis reports with an interactive timeline of flagged segments.
5. **FR-6.5:** The system shall provide a video player for side-by-side comparison of original and sanitized content.

2.3 Non-Functional Requirements

2.3.1 Reliability

1. **REL-1:** The system shall gracefully handle corrupted or unreadable video files by logging the error and continuing with the next file in the queue, without crashing.
2. **REL-2:** The system shall persist processing state (via `metadata.json`) such that a failed job can be inspected post-mortem without data loss.
3. **REL-3:** The system shall target a Mean Time Between Failures (MTBF) of $\geq$100 processing hours under normal operating conditions.

2.3.2 Usability

1. **USE-1:** Parents with no technical background shall be able to upload, process, and view results within 5 minutes of first launch without external documentation.
2. **USE-2:** The system shall provide clear error messages in plain language when validation or processing fails.
3. **USE-3:** The dashboard shall present sensitivity controls as intuitive visual elements (sliders, toggles) rather than requiring numeric input.

2.3.3 Performance

1. **PER-1:** The system shall process video content at 2–3$\times$ faster than real-time playback speed on a machine with a dedicated GPU (NVIDIA RTX 3050 or better).
2. **PER-2:** Frame preprocessing (resize + normalization) shall complete within 5 ms per frame on average.
3. **PER-3:** The GUI shall remain responsive (no freezes) during background video processing by offloading computation to worker threads.

2.3.4 Security

1. **SEC-1:** All video processing shall occur locally on the user's machine; no video data shall be transmitted over the network.
2. **SEC-2:** Sensitivity configuration and processing history shall be stored locally with user-level file system permissions.
3. **SEC-3:** The system shall not retain raw video data after processing is complete unless explicitly configured by the user.

2.4 Domain Model

Figures 2.2 present the high-level domain entities and their relationships.

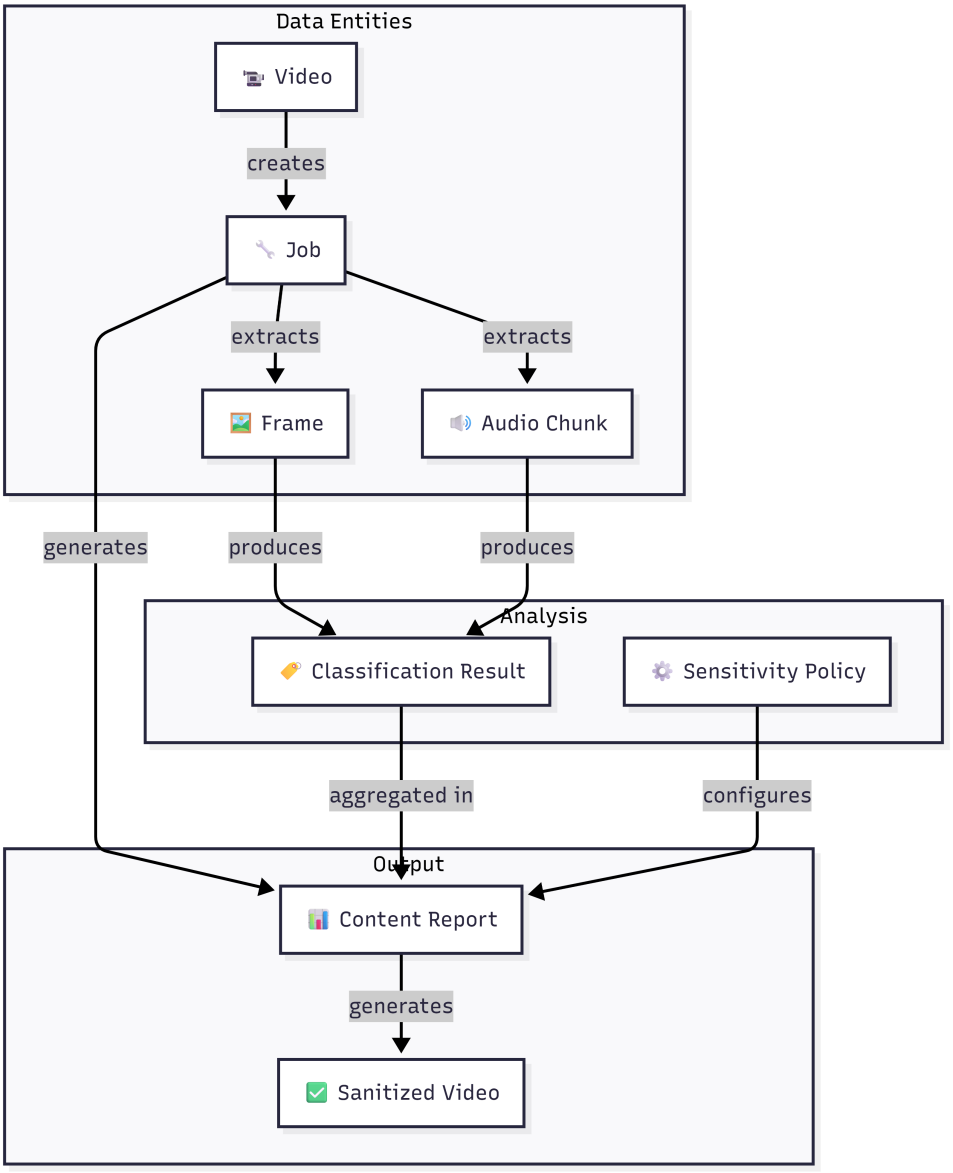

Figure 2.2: Domain Model – Key Entities and Relationships

Chapter 3

System Design

This chapter presents the software design of SafeToon, including the architectural style, design methodology, and the required design models. SafeToon follows an **Object-Oriented Development Approach** using Python 3.13 with a **Layered Architecture** pattern that cleanly separates presentation, business logic, deep learning inference, and data access concerns.

Design Methodology: Object-Oriented Programming (OOP) was selected because SafeToon's modules map naturally to discrete objects with well-defined interfaces (e.g., `VideoValidator`, `JobManager`, `FrameExtractor`). Python's dataclass and module system support clean OOP decomposition with minimal boilerplate.

Architectural Style: A **Layered (Multi-tiered) Architecture** was chosen to enforce separation of concerns. Each layer communicates only with its adjacent layers, enabling independent development, testing, and replacement of components. The parallel processing of visual and audio streams is achieved through Python's threading capabilities within the orchestration layer.

3.1 Architectural Design

SafeToon's architecture comprises four distinct layers as shown in Figure 3.1.

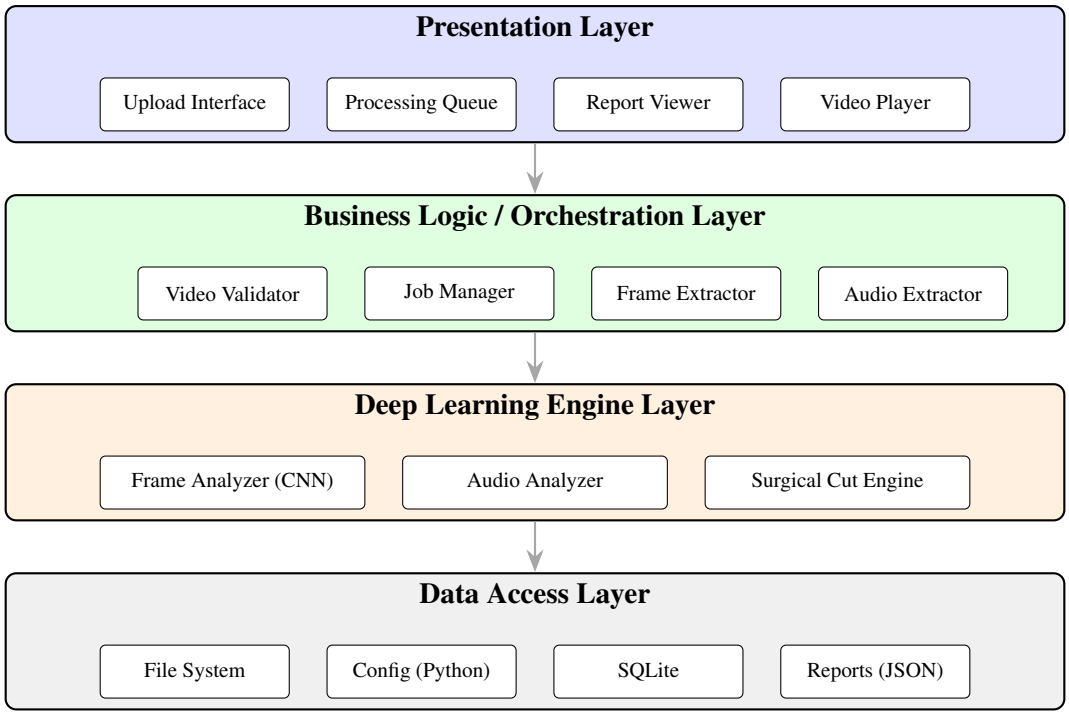

Figure 3.1: Layered System Architecture (TikZ native)

Layer Descriptions:

1. **Presentation Layer:** The Parent Dashboard (planned for PyQt6) provides the user interface for uploading videos, configuring sensitivity, viewing reports, and playing sanitized output.
2. **Business Logic Layer:** The Ingestion Pipeline orchestrates the entire processing flow. It validates input, manages jobs, and coordinates frame and audio extraction using the components built in Iteration 1.
3. **Deep Learning Engine Layer:** Contains the CNN-based Frame Analyzer (to be trained in Iteration 2), the Audio Analyzer (Whisper + NLP), and the Surgical Cut Engine that applies censorship based on classification results.
4. **Data Access Layer:** Manages all persistent storage including the file system (raw videos, extracted frames, audio chunks), configuration files, local database (SQLite for settings and logs), and generated reports.

3.2 Design Models

The following design models document SafeToon's structure and behavior using an Object-Oriented development approach. For each diagram an image-based version are provided.

3.2.1 Activity Diagram

The activity diagram in Figure 3.2 shows the process flow for end-to-end video processing, highlighting the parallel execution of frame and audio analysis pipelines.

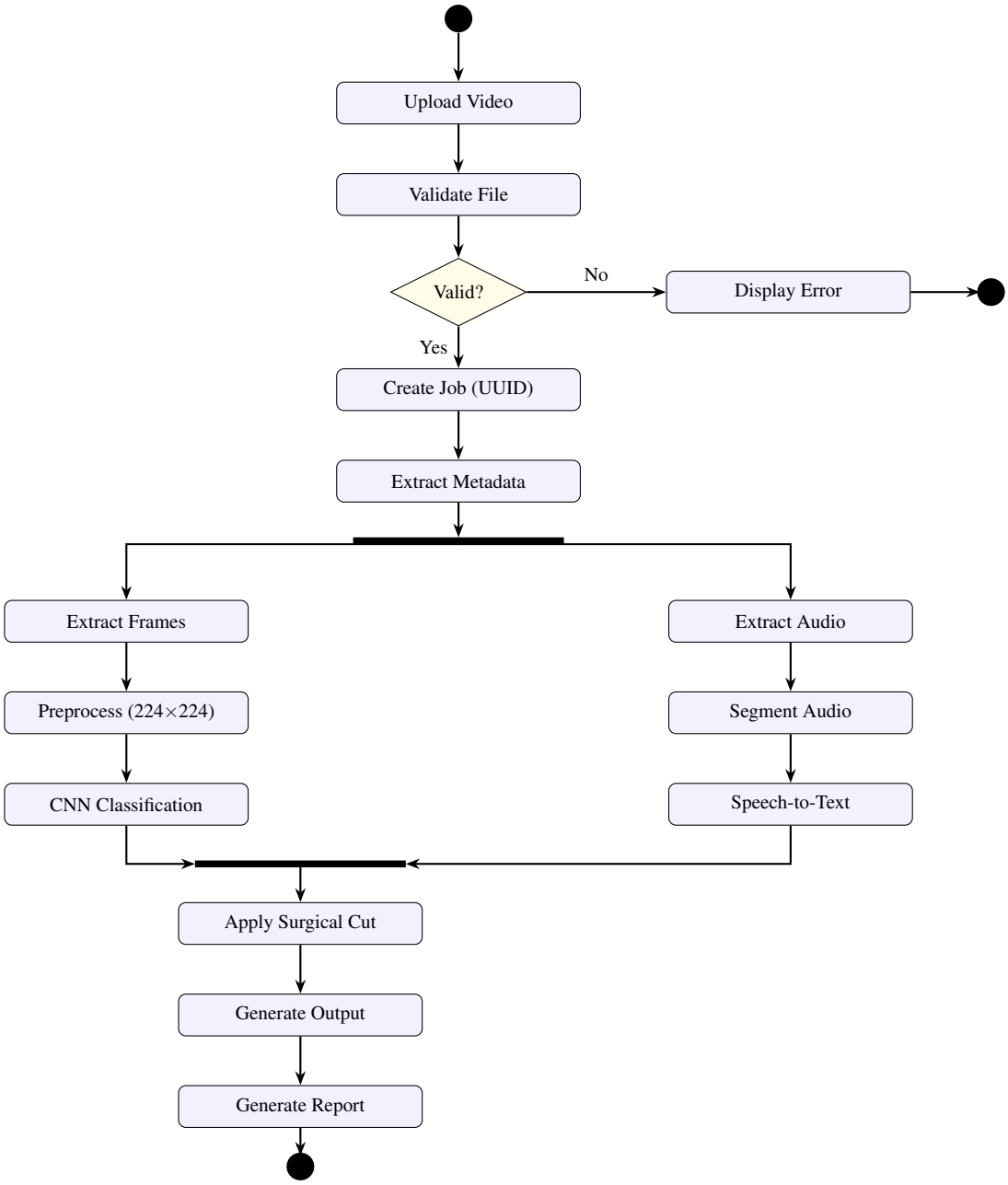

Figure 3.2: Activity Diagram – End-to-End Processing Flow

3.2.2 Class Diagram

The class diagram in Figure 3.3 shows the key classes implemented in Iteration 1 and their relationships.

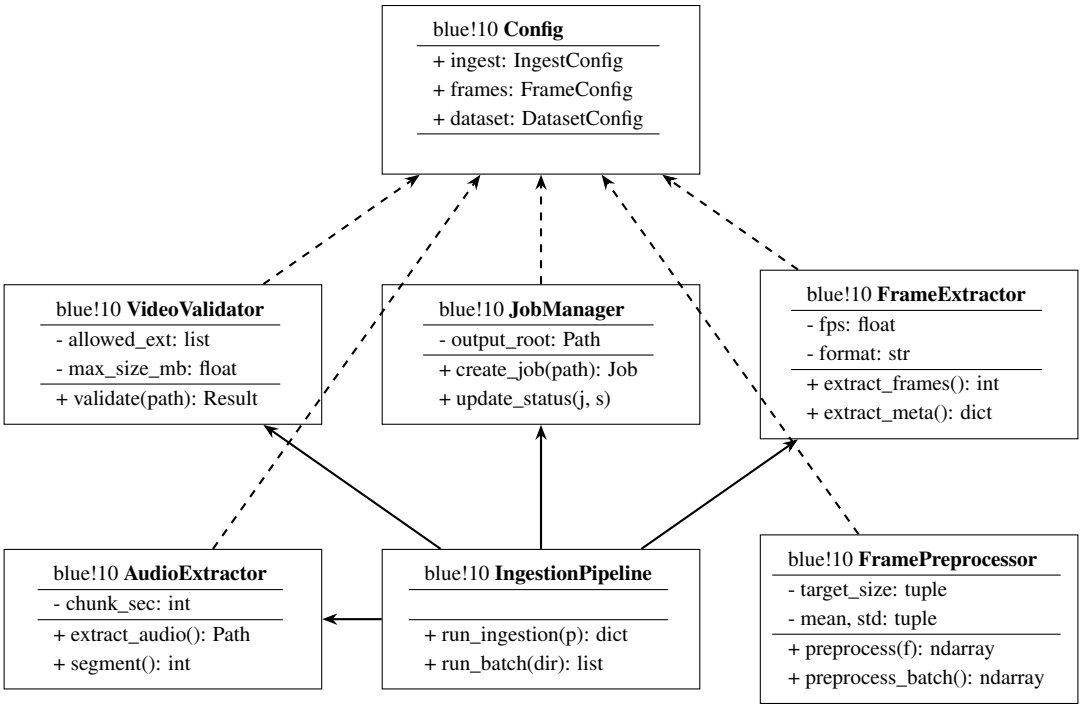

Figure 3.3: Class Diagram – Iteration 1 Components

3.2.3 Sequence Diagram (System Level)

The sequence diagram in Figure 3.4 shows the system-level interaction when a parent submits a video for processing.

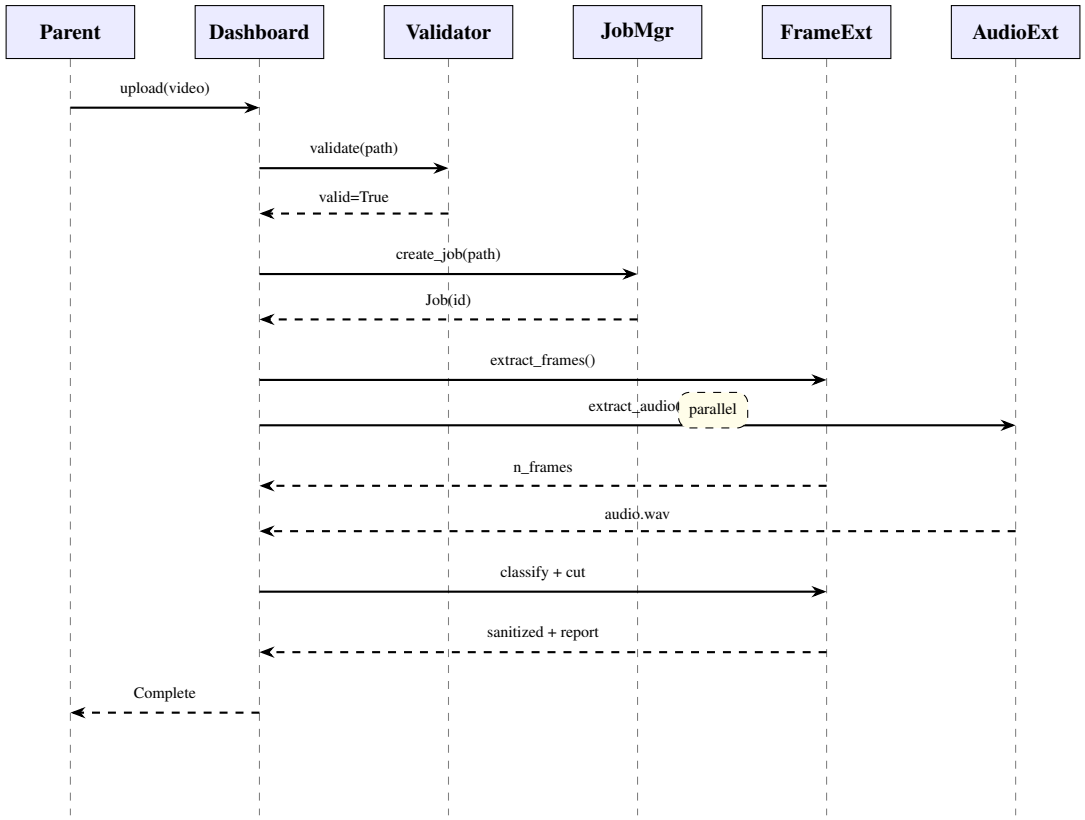

Figure 3.4: Sequence Diagram – Video Processing Flow

3.3 Data Design

SafeToon does not use a traditional relational database for its core pipeline. Instead, it uses **structured JSON** for per-job metadata and results, a **CSV** index for frame-to-job mapping, and **SQLite** (planned for Iteration 4) for user settings and processing history. This design was chosen because the processing pipeline is batch-oriented and file-centric, making JSON a natural fit for per-job state tracking.

3.3.1 Job Metadata Schema (JSON)

Each ingested video creates a job folder containing a `metadata.json` file. The schema is as follows:

Listing 3.1: metadata.json – Per-Job Data Schema

```
{
  "job_id":        "UUID string",
  "video_path":    "absolute path to source video",
  "created_at":    "ISO 8601 timestamp (UTC)",
```

```
  "status":         "created | running | done | error",
  "frames_dir":     "path to outputs/<job_id>/frames/",
  "audio_dir":      "path to outputs/<job_id>/audio/",
  "metadata_path": "path to this file",
  "video_metadata": {
    "duration":     "float (seconds)",
    "fps":          "float",
    "width":        "int (pixels)",
    "height":       "int (pixels)",
    "has_audio":    "boolean",
    "format":       "string (e.g., 'mp4')",
    "size_bytes":   "int"
  },
  "n_frames":        "int (frames extracted)",
  "n_audio_chunks": "int (audio segments created)",
  "audio_path":      "path to audio.wav",
  "error":           "string (if status = error)"
}
```

3.3.2 Frame Index Schema (CSV)

The frame index maps every extracted frame to its source video and job for traceability:

Table 3.1: Frame Index Schema (`frame_index.csv`)

Column	Type	Description
`frame_path`	string	Absolute path to the frame image file
`job_id`	string (UUID)	Job identifier linking to `metadata.json`
`video_path`	string	Path to the original source video
`frame_number`	int	Sequential frame index (0-based)

3.3.3 Dataset Tracking Schema (CSV)

The dataset management layer uses a tracking CSV for data provenance:

Table 3.2: Dataset Index Schema (`dataset_index.csv`)

Column	Type	Description
`video_id`	string	Unique identifier for the video clip
`source`	string	Origin of the video (e.g., Kaggle, YouTube)
`label`	string	Classification label: `violence`, `erotic`, or `normal`
`split`	string	Dataset partition: `train`, `val`, or `test`
`duration_s`	float	Video duration in seconds
`fps`	float	Native frame rate
`resolution`	string	Width×Height (e.g., "1920x1080")
`notes`	string	Annotator comments or flags (e.g., "review")

3.3.4 Output Folder Structure

The file system layout per job serves as the primary data organization mechanism:

Listing 3.2: Per-Job Output Directory Structure

```
outputs/
  <job_id>/
    frames/
      frame_000000.jpg
      frame_000001.jpg
      ...
    audio/
      audio.wav
      chunk_0000.wav
      chunk_0001.wav
      ...
    metadata.json
```

This file-centric approach ensures that all artifacts for a given processing job are self-contained, portable, and independently inspectable without requiring database connectivity.

Bibliography

[1] Akari Ishikawa, Sandra Avila, and Edson Bollis. What are kids watching at youtube? elsagate detection through deep neural networks. *Revista dos Trabalhos de Iniciação Científica da UNICAMP*, 27:1–1, 2019.

[2] Kostantinos Papadamou et al. Disturbed youtube for kids: Characterizing and detecting inappropriate videos targeting young children. In *Proceedings of the AAAI Conference on Web and Social Media*, volume 14, 2020.

[3] Muhammad Rizwan et al. Deep learning based preliminary pipelined architecture to mitigate elsagate: A kid-friendly visual content classification problem. *Applied Sciences*, 12(3):1607, 2022.

[4] Yixin Tang. Animenet: A deep learning approach for detecting violence and eroticism in animated content. *Computers, Materials & Continua*, 77(1):867–891, 2023.